The setup code is shared across specs, but the individual Sandwich instance is not. Every example gets its own sandwich. That means you can add toppings (as you do in the second spec) with the confidence that the changes won't affect other examples.

Now that you've moved the setup code to a common place, you can remove the repeated code from your examples. You'll need to use @sandwich in place of sandwich:

01-getting-started/05/spec/sandwich_spec.rb

```
it 'is delicious' do
  taste = @sandwich.taste

  expect(taste).to eq('delicious')
end

it 'lets me add toppings' do
  @sandwich.toppings << 'cheese'
  toppings = @sandwich.toppings

  expect(toppings).not_to be_empty
end
```

Once you've made the changes, run your new specs. They should all pass, as before.

Hooks are great for running common setup code that has real-world side effects. If you need to clear out a test database before each example, a hook is a great place to do so.

They also work fine for stashing your test objects in instance variables, as we've done here. However, instance variables have a few drawbacks.

First, if you misspell @sandwich, Ruby will silently return nil instead of aborting with a failure right away. The result is typically a confusing error message about code that's far away from the typo.

Second, to refactor your specs to use instance variables, you've had to go through the entire file and replace sandwich with @sandwich.

Finally, when you initialize an instance variable in a before hook, you pay the cost of that setup time for all the examples in the group, even if some of them never use the instance variable. That's inefficient and can be quite noticeable when setting up large or expensive objects.

Let's try a different approach. Undo the changes you made for the before hook so that your file looks like it did before:

01-getting-started/04/spec/sandwich_spec.rb

```
Sandwich = Struct.new(:taste, :toppings)
```